

Electoral Integrity Analyzer

Glosario de términos

Los siguientes términos se usan con el significado específico indicado a lo largo de este documento. Los términos de dominio electoral siguen la terminología del NIST Common Data Formats (National Institute of Standards and Technology, 2024a); los términos de ciberseguridad siguen el vocabulario del NIST SP 800-218 (Scarfone et al., 2022).

Término	Definición en el contexto de este documento
Acta electoral	Documento oficial que registra los totales de votos de una mesa al cierre de la jornada. En el sistema existe en dos versiones: el acta firmada (versión de referencia) y el acta publicada (versión difundida). Cualquier discrepancia entre ambas es una alerta de integridad.
Alerta	Registro generado por el sistema cuando una condición de anomalía (determinística o estadística) se cumple para una entidad. Cada alerta incluye: código, severidad, tipo y ID de la entidad afectada, y evidencia rastreable en los datos.
Anomalía inyectada	Registro creado deliberadamente durante la generación sintética de datos que viola una regla o es estadísticamente atípico. Constituye la base del ground truth para evaluar el desempeño del sistema.
Ballot stuffing (relleno de urnas)	Fraude electoral que consiste en insertar votos adicionales a los legítimamente emitidos para inflar artificialmente la participación o los totales de un candidato en una mesa. Se manifiesta como participación estadísticamente atípica respecto al conjunto.
Boleta	Documento físico o registro digital mediante el cual el votante ejerce su derecho al sufragio. Cada circunscripción tiene un tipo de boleta asignado; usar una boleta de tipo incorrecto (CIR-03) es una irregularidad procedimental.
Carga anómala de operador	Número de acciones registradas por un usuario del sistema que supera en tres o más desviaciones estándar la media del conjunto de operadores. Señal de posible automatización, cuenta comprometida o abuso de acceso.
Check-in electoral	Registro de la asistencia del votante a su mesa asignada el día de la jornada electoral. El check-in es el mecanismo que habilita el ejercicio del voto e impide el doble sufragio. Se modela en el dataset <code>03_registro_sufragio.csv</code> .
Circunscripción electoral	División geográfica electoral que determina qué candidatos o cargos aparecen en la boleta de cada votante. Votar en una circunscripción diferente a la autorizada (CIR-02) implica que el voto afecta una elección para la que el ciudadano no tiene derecho, distorsionando la representación.
Clasificación objetiva de boleta	Etiqueta de validez asignada a cada boleta durante la generación sintética de datos, que representa el estado correcto según las reglas de escrutinio (válida, nula o blanca). Sirve como referencia para detectar discrepancias con la clasificación realizada por el jurado humano (MAN-01, MAN-02).
Cobertura (recall)	Proporción de anomalías del ground truth que el sistema detectó correctamente. Cobertura = coincidencias / alertas esperadas. Mide qué fracción del fraude conocido fue capturada.

Término	Definición en el contexto de este documento
Detector	Módulo de software que implementa el protocolo <code>AlertDetector</code> y encapsula las reglas de detección de una etapa electoral. Recibe los datasets como entrada y devuelve un DataFrame de alertas.
Entidad	Unidad sobre la que se acumula el score de riesgo. Las entidades del sistema son: votante (<code>voter_id</code>), mesa (<code>mesa_id</code>), usuario del sistema (<code>usuario_id</code>) y archivo de acta (<code>archivo_id</code>). Cada alerta se asocia a una entidad, y el score agrega todas las alertas de la misma entidad.
Escrutinio manual	Proceso de clasificación de boletas físicas realizado por un jurado humano para determinar si cada boleta es válida, nula o en blanco. En el sistema se modela en el dataset <code>04_clasificacion_votos_manual.csv</code> .
Ground truth (conjunto de referencia)	Conjunto de alertas esperadas, construido a partir de las anomalías inyectadas durante la generación sintética, que sirve como referencia para evaluar el desempeño del sistema. Se almacena en <code>08_alertas_esperadas.csv</code> .
Hash criptográfico	Valor de longitud fija generado por una función matemática (SHA-256, NIST FIPS 180-4) a partir del contenido de un archivo. Cualquier modificación del archivo, por mínima que sea, produce un hash diferente. El sistema compara el hash original registrado al cierre con el hash actual para detectar alteraciones.
Jornada electoral	Período oficial de votación, definido en el sistema con horario 08:00–16:00. Cualquier check-in fuera de este intervalo activa la alerta SUF-03.
Mesa electoral	Puesto de votación físico al que cada votante está asignado según el padrón. Cada mesa tiene un número de electores habilitados, un operador asignado y produce un acta de resultados al cierre.
Outlier (valor atípico)	Registro cuyo valor se aleja más de tres desviaciones estándar de la media del conjunto ($ Z \geq 3$) en una variable cuantitativa analizada. En el contexto de este sistema, un outlier no es necesariamente fraudulento pero requiere revisión por su alejamiento estadístico del comportamiento del conjunto.
Padrón electoral	Registro oficial de ciudadanos habilitados para votar en una elección determinada, incluyendo datos de identidad, estado legal, estado documental y mesa asignada. En el sistema se modela en <code>01_padron_votantes.csv</code> .
Pipeline de análisis	Secuencia de pasos de procesamiento que transforma los datasets de entrada en alertas y métricas de salida. Implementado en <code>src/pipeline/analysis_pipeline.py</code> , invoca los seis detectores en orden y consolida sus resultados.
Precisión (precision)	Proporción de alertas detectadas que corresponden a anomalías reales del ground truth. $\text{Precisión} = \text{coincidencias} / \text{alertas detectadas}$. Mide qué fracción de las señales del sistema son correctas.
RBAC (Role-Based Access Control)	Modelo de control de acceso en el que los permisos se asignan a roles, y los usuarios obtienen permisos al ser asignados a un rol. En el sistema, cada rol tiene un conjunto explícito de acciones permitidas (<code>ALLOWED_ACTIONS_BY_ROLE</code>). Una acción fuera de ese conjunto activa la alerta LOG-02.
Score de riesgo	Puntuación acumulada que refleja el riesgo asociado a una entidad, calculada sumando los puntos de todas sus alertas ponderadas por severidad (Crítica=3, Alta=2, Media=1). Permite ordenar entidades por prioridad de revisión.
Semilla aleatoria (RANDOM_SEED)	Valor fijo (42) que inicializa el generador de números pseudoaleatorios usado en la producción de datasets sintéticos. Garantiza que toda ejecución del script de

Término	Definición en el contexto de este documento
	generación produce exactamente los mismos datos, haciendo los resultados del análisis reproducibles y auditables.
SoD (Separation of Duties, Separación de funciones)	Principio de control interno que establece que ningún actor individual debe poder completar de forma unilateral un proceso crítico. En el sistema, se verifica que el usuario que modifica un registro no sea el mismo que lo aprueba (LOG-06).
Sufragio / voto registrado	En el padrón (01_padron_votantes.csv), el campo voto_registrado = 1 indica que el sistema registró el ejercicio del sufragio por parte del ciudadano. Su presencia para un ciudadano fallecido (PAD-01), menor (PAD-02) o inhabilitado (PAD-05) activa alertas de padrón de severidad crítica.
Z-score	Medida estadística que expresa cuántas desviaciones estándar se aleja un valor observado de la media de su conjunto. Se calcula como $Z = (x - \mu) / \sigma$. El sistema marca como anomalía estadística todo registro con $ Z \geq 3$.

1. Contexto del Problema

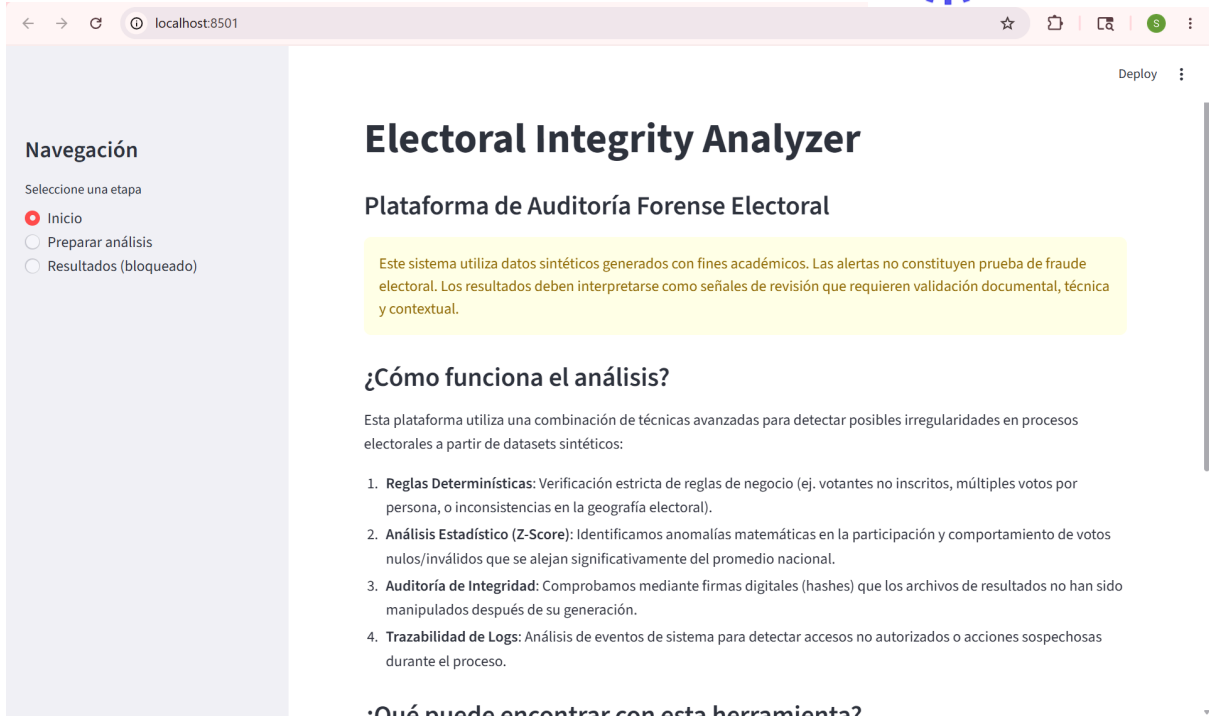
1.1 La integridad electoral como dominio de ciberseguridad

Un proceso electoral genera y transforma datos en al menos seis etapas secuenciales: verificación del padrón, asignación de mesa, registro del sufragio, escrutinio manual de boletas, consolidación de resultados y trazabilidad de operaciones del sistema. En cada etapa, los datos tienen un estado válido definible y transiciones autorizadas explícitas: un votante fallecido no puede ejercer su voto; los resultados de una mesa no pueden exceder el número de electores habilitados; un operador con rol de jurado no puede modificar el padrón.

Cuando estas restricciones no se verifican de forma sistemática, surgen dos categorías de vulnerabilidad. La primera comprende las **violaciones lógicas**, detectables de manera determinística porque infringen una condición que no admite excepciones (votante fallecido con registro de sufragio, sufragantes > electores habilitados, suma de votos $\neq$ total reportado). La segunda comprende las **anomalías estadísticas**, que no violan una regla absoluta pero se alejan de forma significativa del comportamiento del conjunto (una mesa con participación a más de cuatro desviaciones estándar de la media nacional).

El U.S. Election Assistance Commission (2024) identifica como riesgos prioritarios en auditorías post-electorales la inconsistencia entre registros de sufragio y totales de mesa, la falta de trazabilidad de cambios posteriores al cierre electoral, y las discrepancias entre versiones firmadas y publicadas de actas. La detección automatizada y sistemática de estos patrones es la función central del presente proyecto.

Desde la perspectiva de la ingeniería de seguridad, Saltzer y Schroeder (1975) establecieron que los sistemas seguros deben aplicar verificación completa (complete mediation): cada acceso a cada objeto debe ser comprobado frente a la política de autorización vigente. En un proceso electoral, esta verificación no ocurre de forma manual con suficiente cobertura dado el volumen de datos. El presente proyecto implementa esa verificación de forma programática.



1.2 Modelo de amenazas por etapa

La siguiente tabla mapea cada etapa del proceso electoral con su vector de ataque principal y el tipo de anomalía detectable. "Carga anómala de operador" se refiere a un número de acciones del sistema estadísticamente elevado respecto al conjunto, detectado mediante Z-score sobre el log de eventos (ver Glosario). Este modelo de amenazas estructuró el diseño de los detectores.

Etapa electoral	Activo crítico		Vector de ataque principal	Tipo de detección
Padrón y elegibilidad	Registro de votantes	de	Inscripciones inválidas (fallecidos, menores, documentos cancelados)	Determinística
Circunscripción y mesa	Asignación de votante-mesa		Voto fuera de circunscripción autorizada	Determinística
Registro de sufragio	Check-in en jornada	en	Doble voto, registro fuera de horario, carga anómala de operador	Determinística + Z-score
Escrutinio manual	Clasificación de boletas	de	Sesgo de clasificador (invalidación desproporcionada de votos válidos)	Determinística + Z-score
Resultados por mesa	Actas de resultados	de	Totales inconsistentes, participación superior al 100%, outliers estadísticos	Determinística + Z-score
Logs e integridad	Trazabilidad de actas digitales	y	Violaciones de RBAC, cambios post-cierre, alteración de hashes, autovalidación	Determinística

1.3 Objetivos del proyecto

1. Implementar un pipeline de análisis modular que procese datos electorales sintéticos en las seis etapas identificadas y produzca alertas normalizadas con evidencia rastreada.
2. Aplicar reglas determinísticas para detectar violaciones lógicas inequívocas en padrón, sufragio, resultados y trazabilidad de sistema.
3. Aplicar análisis estadístico por Z-score para identificar outliers relativos al conjunto en variables de participación, nulidad y carga de operadores.
4. Modelar controles de ciberseguridad operacional (RBAC, separación de funciones, integridad por hash, monitoreo de intentos fallidos) como alertas auditables con evidencia en datos y código.
5. Cuantificar el riesgo por entidad mediante puntuación ponderada por severidad y generar un ranking que priorice la revisión manual del auditor.
6. Evaluar el desempeño del sistema comparando las alertas detectadas contra el ground truth construido a partir de las anomalías inyectadas.

1.4 Alcance y limitaciones

El sistema opera exclusivamente sobre datos sintéticos generados con semilla aleatoria fija (`RANDOM_SEED = 42`), garantizando reproducibilidad pero implicando que los patrones de anomalías no necesariamente reflejan la distribución de irregularidades en procesos electorales reales. Los hashes en los datasets se modelan como evidencia conceptual de integridad; no se implementa infraestructura de clave pública (PKI) real ni firma digital criptográfica. El sistema no aplica técnicas de aprendizaje automático; las únicas técnicas de detección son reglas determinísticas y Z-score. Toda alerta debe interpretarse como señal de revisión que requiere validación documental, técnica y contextual por parte de un auditor.

2. Metodología

2.1 Marco de referencia

El diseño del sistema toma como referencia los siguientes estándares:

- **NIST Common Data Formats (CDF) — NIST GCR 24-058** (National Institute of Standards and Technology, 2024a): define el modelo de datos para registros de votantes, actas de urna y resultados electorales. Sirvió como referencia para estructurar los campos de los datasets sintéticos.
- **EAC Post-Election Tabulation Audit Guide** (U.S. Election Assistance Commission, 2024): establece qué métricas y verificaciones debe cubrir una auditoría post-electoral, incluyendo la comparación entre registros de sufragio y totales de mesa, y la trazabilidad de cambios en actas.
- **NIST SSDF SP 800-218** (Scarfone et al., 2022): guía de desarrollo seguro de software. Orienta las decisiones de arquitectura: validación de entradas antes de procesar, separación de capas y pruebas automatizadas.
- **NIST FIPS 180-4** (National Institute of Standards and Technology, 2015): estándar para funciones de hash seguro (SHA-256). Fundamenta el uso de hashes como mecanismo de verificación de integridad de actas en el módulo de logs e integridad.

- **NIST SP 800-92** (Kent & Souppaya, 2006): guía de gestión de logs de seguridad. Informa el diseño del dataset `06_logs_eventos.csv` y los controles de auditoría implementados sobre él.

2.2 Datos sintéticos y reproducibilidad

2.2.1 Decisión de no usar datos reales

El proyecto opera exclusivamente sobre datos sintéticos. Esta decisión responde al principio de **privacidad por diseño** (privacy by design), según el cual la protección de datos personales debe incorporarse como decisión arquitectónica desde el inicio del desarrollo, no como medida posterior (Cavoukian, 2009). Procesar datos electorales reales impondría obligaciones de tratamiento de datos personales que exceden el alcance académico del proyecto e introduciría riesgo de exposición de información sensible de ciudadanos.

2.2.2 Semilla aleatoria fija

Los nueve datasets se generan mediante el script `scripts/generate_synthetic_data.py` con `RANDOM_SEED = 42`. Esta decisión responde a un requisito de **reproducibilidad auditable**: una herramienta de análisis forense que produce resultados distintos en ejecuciones sucesivas con la misma entrada no puede utilizarse como evidencia verificable. Con semilla fija, cualquier evaluador obtiene exactamente los mismos hallazgos con el mismo conjunto de datos, lo que permite contraste independiente de los resultados.

2.2.3 Estructura de los datasets

Se generaron nueve datasets interrelacionados que modelan el ciclo electoral completo. La siguiente tabla describe el contenido y las anomalías inyectadas en cada uno.

#	Archivo	Contenido principal	Anomalías inyectadas
01	<code>01_padron_votantes.csv</code>	Censo oficial (edad, estado documental, condición legal)	Fallecidos con voto, menores habilitados, documentos cancelados, edades imposibles, documentos duplicados
02	<code>02_asignacion_mesas.csv</code>	Votante ↔ mesa y circunscripción autorizada	Base para detectar voto fuera de circunscripción
03	<code>03_registro_sufragio.csv</code>	Check-in en jornada (hora, operador, mesa)	Doble sufragio, check-ins fuera de horario electoral
04	<code>04_clasificacion_votos_manual.csv</code>	Clasificación de boletas por jurado	Discrepancias entre clasificación objetiva (referencia sintética) y clasificación del jurado
05	<code>05_resultados_mesa.csv</code>	Totales agregados por mesa	Sufragantes > habilitados, totales inconsistentes, outliers estadísticos de participación
06	<code>06_logs_eventos.csv</code>	Trazabilidad de acciones de sistema (hash antes/después)	Violaciones de RBAC, cambios post-cierre, intentos fallidos, autovalidación (SoD)

#	Archivo	Contenido principal	Anomalías inyectadas
07	07_integridad_archivos.csv	Hashes de actas y reportes publicados	Hash actual $\neq$ hash original (tampering de actas), discrepancias entre versión firmada y publicada
08	08_alertas_esperadas.csv	Ground truth para evaluación	N/A — etiquetas de referencia de anomalías inyectadas
09	usuarios_sistema.csv	Usuarios, roles y permisos del sistema	N/A — catálogo de identidades y permisos

2.3 Técnica 1: Reglas determinísticas

Las reglas determinísticas verifican condiciones lógicas de cumplimiento obligatorio. Cuando la condición se viola, la anomalía es inequívoca e independiente del comportamiento del resto del conjunto de datos. Se aplican cuando la irregularidad no admite excepciones estadísticas: no existe ningún umbral que haga aceptable que una persona fallecida aparezca con voto registrado, o que la suma de votos de una mesa no cuadre con el total reportado en el acta.

Cada regla tiene asignada una severidad según el siguiente criterio formal:

- **Crítica** — la condición viola una restricción absoluta cuya única causa plausible es fraude activo o manipulación directa del sistema. El hallazgo invalida la entidad afectada por sí solo. No existe una explicación operacional legítima.
- **Alta** — la condición viola una restricción procedimental o de control que puede tener explicaciones alternativas: error administrativo, fallo de sistema o error humano. El hallazgo requiere corroboración documental o técnica antes de concluir fraude.
- **Media** — la condición es una señal de alerta que solo es significativa dentro de un patrón más amplio. Por sí sola no justifica acción correctiva.

La siguiente tabla presenta los 27 códigos de alerta del sistema con su condición de disparo, severidad asignada y justificación de esa clasificación.

Código	Etap	Condición de disparo	Sev.	Justificación de la severidad
PAD-01	Padrón	<code>fecha_defuncion < fecha eleccion AND voto_registrado = 1</code>	Crít.	No existe causa operacional para que un fallecido tenga check-in. Implica manipulación del padrón o del registro.
PAD-02	Padrón	<code>edad < 18 AND voto_registrado = 1</code>	Crít.	La mayoría de edad es condición legal absoluta. No admite excepción operacional.
PAD-03	Padrón	<code>edad > 115 O edad calculada desde fecha_nacimiento es imposible</code>	Alta	Puede ser error de ingreso de datos (año de nacimiento incorrecto). Requiere verificación documental antes de concluir fraude.
PAD-04	Padrón	<code>estado_documento ∈ {cancelado, inconsistente} AND voto_registrado = 1</code>	Alta	El estado del documento puede tener rezago administrativo en la

Código	Etap	Condición de disparo	Sev.	Justificación de la severidad
				base de datos. Requiere verificación en el registro civil.
PAD-05	Padrón	condicion_legal = inhabilitado AND voto_registrado = 1	Crít.	La inhabilitación legal es una restricción judicial explícita. No admite excepción operacional.
PAD-06	Padrón	numero_documento duplicado en el padrón	Alta	La duplicación puede ser error del proceso de inscripción. Requiere auditoría del registro para descartar inscripción múltiple deliberada.
SUF-01	Sufragio	COUNT(voter_id) > 1 en registro_sufragio	Crít.	No existe escenario legítimo para dos check-ins del mismo votante. Infla directamente el conteo de la mesa.
SUF-02	Sufragio	voter_id no habilitado en padrón con check-in registrado	Crít.	Una persona fuera del padrón que vota indica que el control de elegibilidad fue eludido por completo. El voto es inválido.
SUF-03	Sufragio	hora_checkin fuera del horario electoral (08:00–16:00)	Alta	Puede ser error de reloj del sistema o excepción autorizada no registrada. Requiere verificación técnica del timestamp.
CIR-01	Circunscripción	mesa_voto ≠ mesa_asignada	Alta	Votar en otra mesa de la misma circunscripción es un error procedimental: el voto puede seguir siendo válido para el mismo candidato. El impacto electoral puede ser bajo si la circunscripción es la misma.
CIR-02	Circunscripción	circunscripcion_voto ≠ circunscripcion_autorizada	Crít.	Votar en circunscripción incorrecta significa que la boleta afecta una elección para la que el votante no tiene derecho. Distorsiona la representación, no solo el procedimiento.
CIR-03	Circunscripción	Tipo de boleta utilizada ≠ tipo autorizado para circunscripción	Alta	Puede ser error de distribución logística de boletas en la mesa. Requiere verificación presencial.
MAN-01	Escrutinio	clasificacion_objetiva = válido AND clasificacion_jurado = inválido	Alta	El jurado puede haber clasificado la boleta por error ante marcas ambiguas. Sin revisión del documento físico, no es posible distinguir error humano de sesgo deliberado.
MAN-02	Escrutinio	clasificacion_objetiva = inválido AND	Alta	Mismo razonamiento que MAN-01 en dirección inversa. Requiere revisión física de la boleta.

Código	Etapas	Condición de disparo	Sev.	Justificación de la severidad
		<code>clasificacion_jurado = válido</code>		
RES-01	Resultados	<code>sufragantes_registrados > electores_habilitados</code> (conteo crudo)	Crít.	Imposibilidad aritmética en los conteos crudos: no puede haber más personas que votaron que personas habilitadas para votar. Ningún escenario operacional lo explica.
RES-02	Resultados	<code>participacion_pct > 1.0</code> (campo calculado > 100%)	Crít.	El campo derivado contiene un valor imposible, lo que indica que fue calculado incorrectamente o modificado de forma independiente a los conteos crudos. Detecta manipulación del campo sin alterar RES-01.
RES-03	Resultados	$\Sigma(\text{votos candidatos} + \text{nulos} + \text{blancos}) \neq \text{total_reportado}$	Crít.	La suma de componentes no cuadra con el total del acta. Es el indicador más directo de falsificación del acta de resultados.
INT-01	Integridad	<code>hash_actual</code> $\neq$ <code>hash_original</code> (dataset 07)	Crít.	Un hash distinto al registrado al cierre prueba que el archivo fue modificado después. Toda modificación post-registro de un acta es alteración de evidencia oficial.
INT-02	Integridad	<code>reporte_publicado_total</code> $\neq$ <code>reporte_firmado_total</code>	Crít.	El documento publicado difiere del firmado. Constituye manipulación de la información electoral comunicada al público.
INT-03	Integridad	<code>timestamp_publicacion</code> < <code>timestamp_firma</code>	Alta	La secuencia incorrecta puede ser un error de sincronización entre relojes de servidores distintos. Requiere verificación técnica del sistema de timestamps.
INT-04	Integridad	Archivo modificado después del cierre sin autorización registrada	Alta	La modificación puede haber tenido una autorización cuyo registro falló por error del sistema de logging. Requiere auditoría cruzada de logs y autorizaciones.
LOG-01	Logs	Modificación registrada después del cierre sin aprobación (dataset 06)	Crít.	Cuando el logging funciona correctamente, una modificación post-cierre sin aprobación es evidencia directa de alteración indebida del sistema.
LOG-02	Logs	<code>accion</code> $\notin$ <code>ALLOWED_ACTIONS[rol_usuario]</code> (violación RBAC)	Crít.	Un usuario con acceso fuera de su rol puede alterar cualquier dato dentro del alcance de esa acción. Es

Código	Etap	Condición de disparo	Sev.	Justificación de la severidad
				la puerta de entrada a cualquier otra forma de manipulación.
LOG-03	Logs	Usuario con estado_usuario $\neq$ activo ejecuta una acción	Alta	La cuenta puede haber sido desactivada con un retraso de propagación en el sistema. Requiere verificar el proceso de baja de usuarios.
LOG-04	Logs	Intentos fallidos de login por encima del umbral configurable	Media	Los intentos fallidos son normales en cualquier sistema (contraseña olvidada, error de teclado). Solo son significativos como patrón sostenido en el tiempo, no como evento aislado.
LOG-05	Logs	hash_despues $\neq$ valor esperado en evento no autorizado (dataset 06)	Crít.	Combina dos condiciones simultáneas: (1) la acción no estaba autorizada y (2) el estado del sistema cambió como resultado. Es evidencia de manipulación exitosa, no solo de intento.
LOG-06	Logs	usuario_id que modifica = aprobado_por (violación SoD)	Alta	La violación de SoD indica un control interno roto, pero no prueba por sí sola que la modificación fue fraudulenta. El contenido del cambio debe revisarse para determinar el impacto.

Nota sobre RES-01 y RES-02: Aunque ambas alertas parecen detectar la misma condición (participación superior al 100%), verifican cosas distintas. RES-01 compara los conteos crudos de sufragantes y electores habilitados; RES-02 verifica el campo calculado `participacion_pct`. Un escenario con RES-01 sin RES-02 indica que los conteos están mal pero el campo derivado fue recalculado correctamente. Un escenario con RES-02 sin RES-01 indica que el campo fue modificado directamente sin alterar los conteos subyacentes, lo que es una forma de manipulación más sofisticada.

Nota sobre INT-01 y LOG-05: Ambas verifican hashes, pero en datasets y momentos distintos. INT-01 detecta el estado final de un archivo en `07_integridad_archivos.csv` (el archivo existe y su hash actual no coincide con el original). LOG-05 detecta un evento específico en `06_logs_eventos.csv` donde una acción no autorizada produjo un cambio de hash (el acto de modificación en tiempo real). INT-01 puede dispararse sin LOG-05 si la modificación no quedó registrada en el log; LOG-05 puede dispararse sin INT-01 si el hash fue revertido después del evento.

El catálogo completo del sistema contiene 33 códigos de alerta definidos en `src/domain/alert_types.py`. La tabla anterior presenta 27 códigos con condición determinística; los seis restantes se activan mediante Z-score sobre variables cuantitativas (ver sección 2.4).

2.4 Técnica 2: Análisis estadístico por Z-score

El Z-score se aplica para detectar anomalías relativas: valores que se alejan de forma significativa del comportamiento del conjunto sin que exista una regla fija que los catalogue como fraudulentos. Un outlier estadístico no es necesariamente fraudulento, pero su alejamiento del comportamiento del resto del conjunto lo convierte en candidato prioritario de revisión. La fórmula aplicada es:

$$Z = \frac{x - \mu}{\sigma}$$

donde x es el valor observado, μ es la media del conjunto y σ es la desviación estándar. Se marca como anomalía todo registro donde $|Z| \geq 3$.

2.4.1 Justificación del umbral $Z = 3$

El umbral de tres desviaciones estándar (regla de tres sigma) es estándar en control estadístico de procesos desde Shewhart (1931). En una distribución normal, la probabilidad de que un valor se aleje más de tres desviaciones estándar de la media es $P(|Z| > 3) \approx 0.0027$, es decir, menos del 0.27% de los valores. Con umbral $Z = 2$, el 4.55% de los registros quedarían marcados solo por variabilidad estadística normal, lo que genera un volumen de señales que diluye los hallazgos relevantes. Con umbral $Z = 3$, el sistema marca únicamente los valores en los extremos genuinos de la distribución.

2.4.2 Variables analizadas

Variable	Dataset	Interpretación de outlier ($ Z \geq 3$)
Participación por mesa (participacion_pct)	05_resultados_mesa.csv	Participación extremadamente alta respecto al conjunto: posible ballot stuffing (inserción fraudulenta de votos para inflar la participación de la mesa).
Tasa de votos nulos e inválidos	05_resultados_mesa.csv	Invalidación desproporcionada respecto al promedio de las mesas: posible supresión dirigida de votos válidos.
Concentración del candidato ganador (concentracion_ganador_pct)	05_resultados_mesa.csv	Concentración estadísticamente atípica: señal de coherencia de resultados cuestionable respecto al comportamiento del resto de las mesas.
Eventos por operador (acciones en log)	06_logs_eventos.csv	Exceso de acciones por usuario respecto al promedio del conjunto: posible operador con cuenta vulnerada o proceso automatizado no autorizado.

La implementación en `src/analytics/statistics.py` maneja el caso de varianza cero (conjunto homogéneo): si la desviación estándar del conjunto es cero, la función `zscore_series()` retorna ceros en lugar de intentar una división por cero, evitando fallas silenciosas en conjuntos de datos con baja diversidad.

2.5 Justificación de la selección metodológica

2.5.1 Por qué no se utilizó aprendizaje automático

El proyecto descartó modelos de detección de anomalías no supervisados (Isolation Forest, autoencoders, clustering DBSCAN) a pesar de que el enunciado académico lo permitía. La razón central es la **explicabilidad** en contextos forenses.

En una auditoría electoral, cada hallazgo debe ser justificable ante una comisión. Un resultado del tipo "esta mesa está a 4.2 desviaciones estándar sobre la media de participación" es reproducible, verificable manualmente y comunicable a un no-especialista. Un resultado del tipo "el modelo lo marcó como anomalía porque cayó fuera de su cluster de densidad variable" no ofrece la trazabilidad que requiere un proceso forense. La explicabilidad de los modelos de detección de anomalías es un requisito documentado en el contexto de auditorías de sistemas automatizados de toma de decisiones (Hu et al., 2014).

Adicionalmente, entrenar modelos de detección de anomalías requiere un conjunto de datos etiquetado o histórico que represente el comportamiento normal del proceso electoral. En el alcance de este proyecto, con datos sintéticos generados para una única elección simulada, no se cuenta con esa base histórica. Usar los mismos datos para entrenar y evaluar un modelo no supervisado produciría métricas de evaluación circulares.

2.5.2 Por qué reglas determinísticas y Z-score son técnicas complementarias

Las reglas determinísticas son necesarias cuando la anomalía es absoluta (violación lógica sin excepciones). El Z-score es necesario cuando la anomalía es relativa al contexto del conjunto. Ninguna de las dos técnicas reemplaza a la otra:

- Una participación del 92% en una mesa no activa una regla determinística porque no supera el 100%. Sí puede activar una alerta por Z-score si el resto de las mesas tienen participaciones en torno al 55%.
- Un fallecido con voto registrado no es un outlier estadístico si otros fallecidos también tienen votos. Requiere una regla determinística, no Z-score.

La combinación de ambas técnicas cubre dos vectores complementarios de fraude: el evidente (violación de regla) y el sutil (desviación estadística significativa).

2.6 Función de puntuación de riesgo

Cada alerta contribuye con puntos al score de riesgo de la entidad implicada. Una entidad es la unidad sobre la que se acumula el riesgo: puede ser un votante (`voter_id`), una mesa (`mesa_id`), un usuario del sistema (`usuario_id`) o un archivo de acta (`archivo_id`). La función de puntuación pondera las alertas por su severidad:

$$\text{Score} = \sum(\text{Crítica} \times 3) + (\text{Alta} \times 2) + (\text{Media} \times 1)$$

La ponderación numérica permite ordenar entidades por riesgo acumulado, produciendo un ranking de revisión priorizada. Los umbrales de clasificación son:

Rango de score	Clasificación
0	Sin alerta
1 – 3	Riesgo bajo

Rango de score	Clasificación
4 – 7	Riesgo medio
8 – 12	Riesgo alto
> 12	Revisión prioritaria

El objetivo de esta función no es reemplazar el juicio del auditor, sino reducir el universo de revisión manual de 1,158 alertas individuales a un conjunto acotado de entidades que concentran la mayor parte del riesgo acumulado.

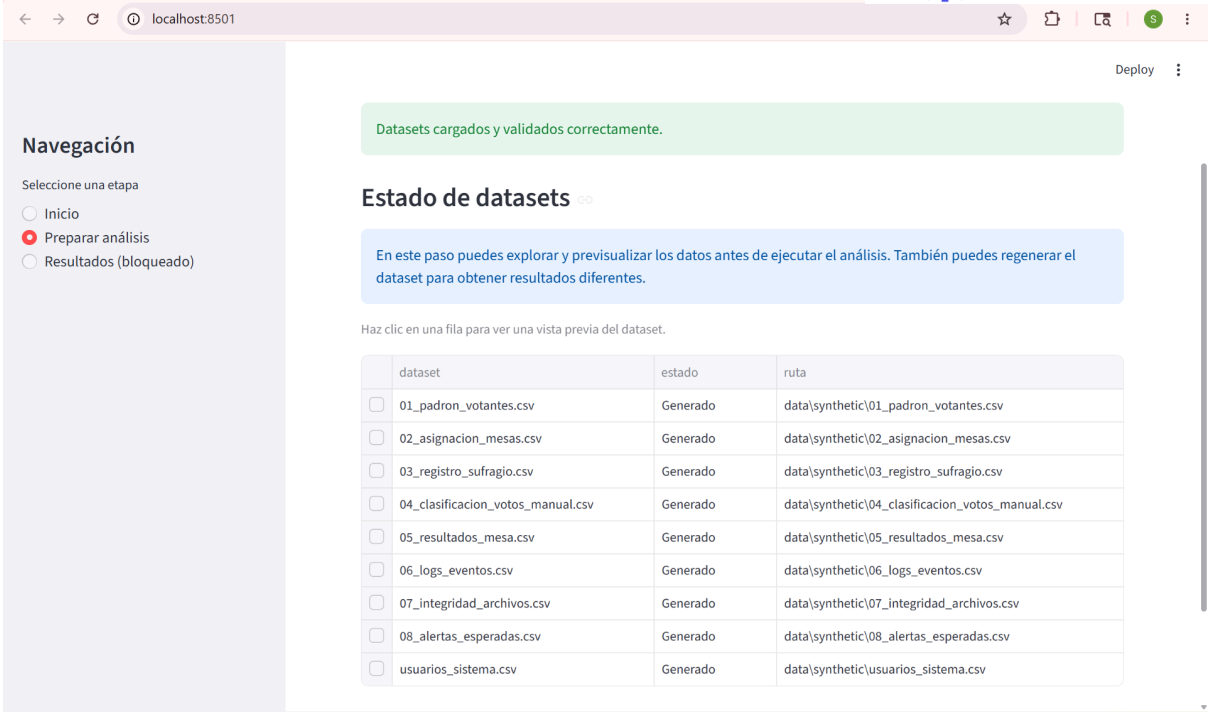
2.7 Evaluación contra ground truth

El ground truth es el conjunto de anomalías inyectadas durante la generación sintética, almacenadas en `08_alertas_esperadas.csv`. Cada registro del ground truth contiene la clave compuesta (`codigo_alerta`, `entidad_tipo`, `entidad_id`) que identifica de forma única la anomalía esperada. Las alertas detectadas se comparan contra este conjunto usando esa misma clave. Las métricas calculadas son:

$$\text{Precisión} = \frac{\text{coincidencias}}{\text{alertas_detectadas}} \quad \text{Cobertura} = \frac{\text{coincidencias}}{\text{alertas_esperadas}}$$

Se utiliza la denominación "aproximada" porque el ground truth sintético etiqueta únicamente las anomalías inyectadas de forma determinística. El Z-score puede generar señales adicionales sobre registros que no forman parte del ground truth pero que son estadísticamente atípicos en el conjunto. Estas señales reducen la precisión calculada sin constituir un error de diseño: en un sistema de priorización de auditoría, el costo de un falso negativo (omitir una mesa con fraude) supera al de un falso positivo (revisar una mesa que resulta limpia).

3. Implementación



← → ↻ localhost:8501

Deploy

Navegación

Seleccione una etapa

- ☐ Inicio
- ☒ Preparar análisis
- ☐ Resultados (bloqueado)

Datasets cargados y validados correctamente.

Estado de datasets

En este paso puedes explorar y previsualizar los datos antes de ejecutar el análisis. También puedes regenerar el dataset para obtener resultados diferentes.

Haz clic en una fila para ver una vista previa del dataset.

	dataset	estado	ruta
☐	01_padron_votantes.csv	Generado	data\synthetic\01_padron_votantes.csv
☐	02_asignacion_mesas.csv	Generado	data\synthetic\02_asignacion_mesas.csv
☐	03_registro_sufragio.csv	Generado	data\synthetic\03_registro_sufragio.csv
☐	04_clasificacion_votos_manual.csv	Generado	data\synthetic\04_clasificacion_votos_manual.csv
☐	05_resultados_mesa.csv	Generado	data\synthetic\05_resultados_mesa.csv
☐	06_logs_eventos.csv	Generado	data\synthetic\06_logs_eventos.csv
☐	07_integridad_archivos.csv	Generado	data\synthetic\07_integridad_archivos.csv
☐	08_alertas_esperadas.csv	Generado	data\synthetic\08_alertas_esperadas.csv
☐	usuarios_sistema.csv	Generado	data\synthetic\usuarios_sistema.csv

3.1 Decisiones de arquitectura y su justificación

3.1.1 Monolito modular por capas

El sistema se implementó como un monolito modular por capas (layered monolith), no como microservicios. Esta decisión responde a que el alcance del proyecto (una herramienta de demostración académica ejecutada localmente) no justifica la complejidad operacional adicional que introduce una arquitectura de microservicios: orquestación de contenedores, comunicación entre servicios, gestión de fallos distribuidos y observabilidad distribuida. Sin embargo, el alcance sí requiere separación de responsabilidades que facilite la adición de nuevos detectores, la auditoría por componente y las pruebas automatizadas. El monolito modular logra ambos objetivos: mantiene la simplicidad operacional mientras impone separación de capas con interfaces bien definidas.

3.1.2 Capas del sistema y responsabilidades

Capa	Módulo	Responsabilidad
Presentación	app.py	Navegación por etapas, estado de sesión (Streamlit), renderizado. Sin lógica de detección.
Carga y validación	src/data_loader/	Lectura de CSV, validación de esquemas, tipos, valores permitidos y llaves referenciales.
Dominio	src/domain/	Catálogo de 33 códigos de alerta con severidad y acción. Esquemas inmutables por dataset.
Detección	src/detectors/	Seis detectores por etapa electoral. Protocolo AlertDetector: detect(datasets) → DataFrame.
Analítica	src/analytics/	Z-score/outliers, puntuación de riesgo, resumen de hallazgos y evaluación vs. ground truth.
Orquestación	src/pipeline/	AnalysisPipeline.run() invoca detectores, consolida alertas y normaliza scores.

Capa	Módulo	Responsabilidad
Visualización	<code>src/visualizations/</code>	Cinco gráficos mediante <code>Plotly</code> mediante <code>build_required_charts()</code> .
Exportación	<code>src/reports/</code>	Generación de tres CSV auditables en <code>data/outputs/</code> .
Datos sintéticos	<code>scripts/</code>	Producción reproducible de los 9 datasets con semilla aleatoria fija.

La separación entre la capa de presentación y la capa de detección responde a un principio de seguridad: si la lógica de detección residiera en `app.py`, un cambio en la interfaz de usuario podría alterar inadvertidamente una regla de detección. Con capas separadas, los cambios son localizables y trazables, lo que es un requisito del principio de separación de privilegios (Saltzer & Schroeder, 1975).

3.1.3 Validación como fail-secure

La capa de carga y validación (`src/data_loader/validators.py`) verifica esquemas, tipos de datos, valores permitidos y consistencia referencial antes de ejecutar cualquier análisis. Si la validación detecta errores, el sistema bloquea el procesamiento.

Esta decisión implementa el principio de fail-secure: un sistema que procesa datos con columnas faltantes o tipos incorrectos puede generar alertas basadas en valores nulos, produciendo confianza falsa en los resultados. Un sistema que rechaza la entrada inválida y detiene el proceso es menos peligroso que uno que falla silenciosamente con datos corruptos. Este principio está alineado con la práctica PW.6 del NIST SSDF ("validar las entradas antes de procesarlas") descrita por Scarfone et al. (2022).

3.2 Patrones de diseño aplicados

Patrón	Aplicación en el proyecto
Strategy + Protocol	Cada detector implementa <code>AlertDetector.detect(datasets) → DataFrame</code> mediante <code>typing.Protocol</code> . Se puede agregar un nuevo detector sin modificar ningún módulo existente.
Factory Method	<code>build_default_detectors()</code> en <code>src/detectors/factory.py</code> devuelve la lista de detectores en orden de ejecución. El pipeline no conoce las clases concretas.
Pipeline / Chain of Responsibility	<code>AnalysisPipeline.run()</code> aplica detectores secuencialmente, agrega los DataFrames de alertas y los pasa por las fases de analítica.
Repository (DAO)	<code>load_datasets()</code> centraliza el acceso a disco. El resto del sistema trabaja con diccionarios de DataFrames y no conoce rutas ni formatos de archivo.
Catalog (DDD)	<code>ALERT_CATALOG</code> define los 33 códigos de alerta como datos de configuración inmutables. Los detectores referencian el catálogo en lugar de codificar metadatos internamente.

3.3 Principios SOLID aplicados

Principio	Aplicación concreta
SRP — Responsabilidad única	Cada módulo tiene una única función: carga, detección, analítica, visualización o exportación. <code>app.py</code> no contiene lógica de detección.
OCP — Abierto/cerrado	Nuevos detectores se agregan implementando el protocolo <code>AlertDetector</code> , sin modificar <code>AnalysisPipeline</code> ni los detectores existentes.
LSP — Sustitución de Liskov	Cualquier detector puede reemplazarse por otro (o por un mock en tests) sin cambiar el comportamiento del pipeline.
ISP — Segregación de interfaces	El contrato del detector expone únicamente <code>detect()</code> . No hay métodos obligatorios no utilizados.
DIP — Inversión de dependencias	<code>app.py</code> depende de <code>AnalysisPipeline</code> (abstracción), no de los detectores concretos. El pipeline depende del protocolo, no de clases específicas.

3.4 Controles de ciberseguridad implementados

La siguiente tabla mapea cada amenaza modelada con el control implementado, las alertas que genera y la evidencia rastreable en datasets y código.

Amenaza	Control	Alertas	Dataset	Módulo
Abuso de rol / escalada de privilegios	RBAC: matriz rol → acciones permitidas	LOG-02	06_logs_ventos.csv	log_integrity_detector.py
Falta de separación de funciones	SoD: quien modifica ≠ quien aprueba	LOG-06	06_logs_ventos.csv	log_integrity_detector.py
Tampering post-cierre	Control de cambios post-cierre + aprobación requerida	LOG-01, INT-04	06, 07	log_integrity_detector.py
Fuerza bruta / acceso no autorizado	Monitoreo de intentos fallidos de login	LOG-04	06_logs_ventos.csv	log_integrity_detector.py
Alteración de evidencia digital	Integridad por hash (original vs. actual)	INT-01, LOG-05	06, 07	log_integrity_detector.py
Publicación sin firma / inconsistente	Control de publicación vs. firma	INT-02, INT-03	07_integridad_archivos.csv	log_integrity_detector.py

3.4.1 Justificación de RBAC

El sistema implementa control de acceso basado en roles (RBAC) conforme al modelo definido por Sandhu et al. (1996): cada usuario tiene asignado un rol (JURADO, ADMIN, AUDITOR, OPERADOR, entre otros), y cada rol tiene un conjunto explícito de acciones permitidas (`ALLOWED_ACTIONS_BY_ROLE` en `log_integrity_detector.py`). La acción de un usuario fuera de ese conjunto activa la alerta LOG-02.

El principio de mínimo privilegio, establecido por Saltzer y Schroeder (1975), prescribe que cada actor debe tener acceso únicamente a los recursos necesarios para su función. En un proceso

electoral, donde los roles son finitos y sus funciones están definidas por reglamento, RBAC es el modelo de control de acceso adecuado: los roles no son dinámicos, no dependen de atributos de contexto variables, y la superficie de error de configuración es menor que en modelos basados en atributos (ABAC) (Hu et al., 2014).

3.4.2 Justificación de la separación de funciones (SoD)

La separación de funciones (Separation of Duties) establece que ningún actor individual debe poder completar un proceso crítico de forma unilateral (Clark & Wilson, 1987). En el sistema, un operador que modifica un registro y también lo aprueba activa la alerta LOG-06. Esta verificación eleva el costo de un ataque interno: comprometer una sola cuenta no es suficiente para manipular resultados sin dejar rastro; se requieren al menos dos cuentas comprometidas actuando en coordinación, lo que amplía la superficie de detección.

3.4.3 Justificación del control de integridad por hash

La integridad de actas digitales se verifica comparando el `hash_original` de cada archivo contra su `hash_actual`. Cualquier modificación del contenido del archivo, incluso de un solo carácter, produce un hash diferente. El estándar aplicado como referencia es SHA-256, especificado en NIST FIPS 180-4 (National Institute of Standards and Technology, 2015).

El proyecto modela hashes como evidencia conceptual de integridad. No se implementa infraestructura PKI real ni firma digital criptográfica, ya que ello requeriría módulos de seguridad de hardware (HSM) y gestión de certificados X.509, fuera del alcance académico. El valor del modelo está en demostrar que el control existe y que el detector verifica la condición: si `hash_actual` $\neq$ `hash_original`, el sistema emite la alerta INT-01 con evidencia rastreable en el log.

3.4.4 Justificación de la defensa en profundidad

El sistema no depende de un único control de seguridad. En términos del principio de defensa en profundidad (defense in depth), múltiples capas de verificación reducen el riesgo de que la falla de un control individual sea suficiente para que una anomalía pase desapercibida (Saltzer & Schroeder, 1975). Un operador con cuenta vulnerada que elude el control RBAC puede ser detectado por el análisis de carga de trabajo por Z-score. Un archivo alterado puede detectarse tanto por la verificación de hash (INT-01) como por la auditoría de eventos post-cierre (LOG-01, INT-04).

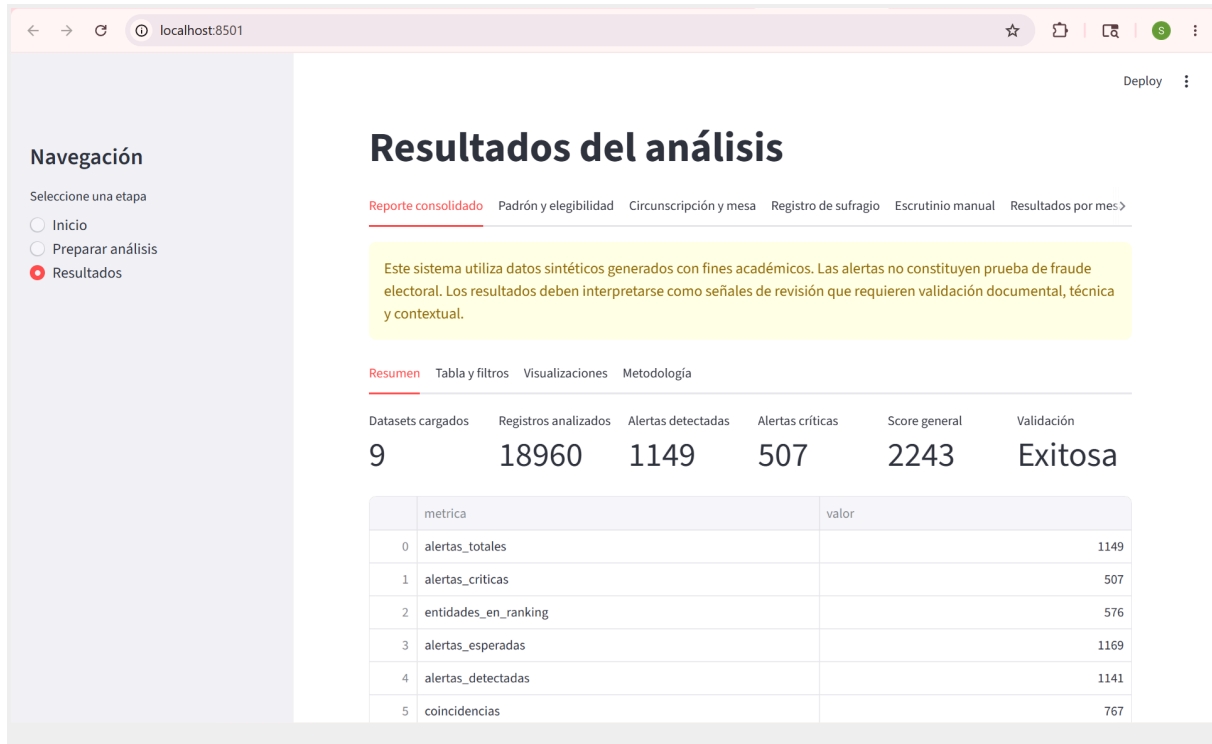
3.5 Alineación con NIST SSDF SP 800-218

El desarrollo refleja prácticas del Secure Software Development Framework (Scarfone et al., 2022), sin afirmar cumplimiento formal. Las decisiones verificables en el código son:

- **PW.6 — Validación de entradas:** `validators.py` aplica validación estructural y referencial antes de ejecutar cualquier análisis.
- **PW.4 — Diseño seguro:** la separación UI/detectores/pipeline reduce la probabilidad de que un cambio en una capa afecte el comportamiento de otra.
- **PW.7 — Identificación de vulnerabilidades mediante pruebas:** la suite `pytest` valida el contrato de columnas de detectores, la ejecución sin excepción del pipeline completo, y que los datasets generados superan la validación de esquemas.

- **PW.2 — Trazabilidad:** semilla aleatoria fija y datasets sintéticos garantizan que la misma entrada produzca el mismo resultado, permitiendo auditoría repetible.

4. Resultados



Resultados del análisis

Reporte consolidado | Padrón y elegibilidad | Circunscripción y mesa | Registro de sufragio | Escrutinio manual | Resultados por mes >

Este sistema utiliza datos sintéticos generados con fines académicos. Las alertas no constituyen prueba de fraude electoral. Los resultados deben interpretarse como señales de revisión que requieren validación documental, técnica y contextual.

Resumen | Tabla y filtros | Visualizaciones | Metodología

Datasets cargados	Registros analizados	Alertas detectadas	Alertas críticas	Score general	Validación
9	18960	1149	507	2243	Exitosa

	metrica	valor
0	alertas_totales	1149
1	alertas_criticas	507
2	entidades_en_ranking	576
3	alertas_esperadas	1169
4	alertas_detectadas	1141
5	coincidencias	767

4.1 Métricas globales del sistema

En la ejecución reproducible con `RANDOM_SEED = 42`, tras regenerar los datasets desde cero, el sistema procesó los nueve datasets y produjo los siguientes resultados:

Métrica	Valor
Total de alertas generadas	1,158
Alertas de severidad crítica	516 (44.6%)
Alertas de severidad alta	627 (54.1%)
Alertas de severidad media	15 (1.3%)
Entidades únicas en ranking de riesgo	582
Alertas esperadas (ground truth)	1,171
Alertas detectadas (base de comparación)	1,150
Coincidencias exactas (clave compuesta)	771
Precisión aproximada	0.6704 (67.0%)
Cobertura aproximada	0.6584 (65.8%)

Resumen Tabla y filtros Visualizaciones Metodología

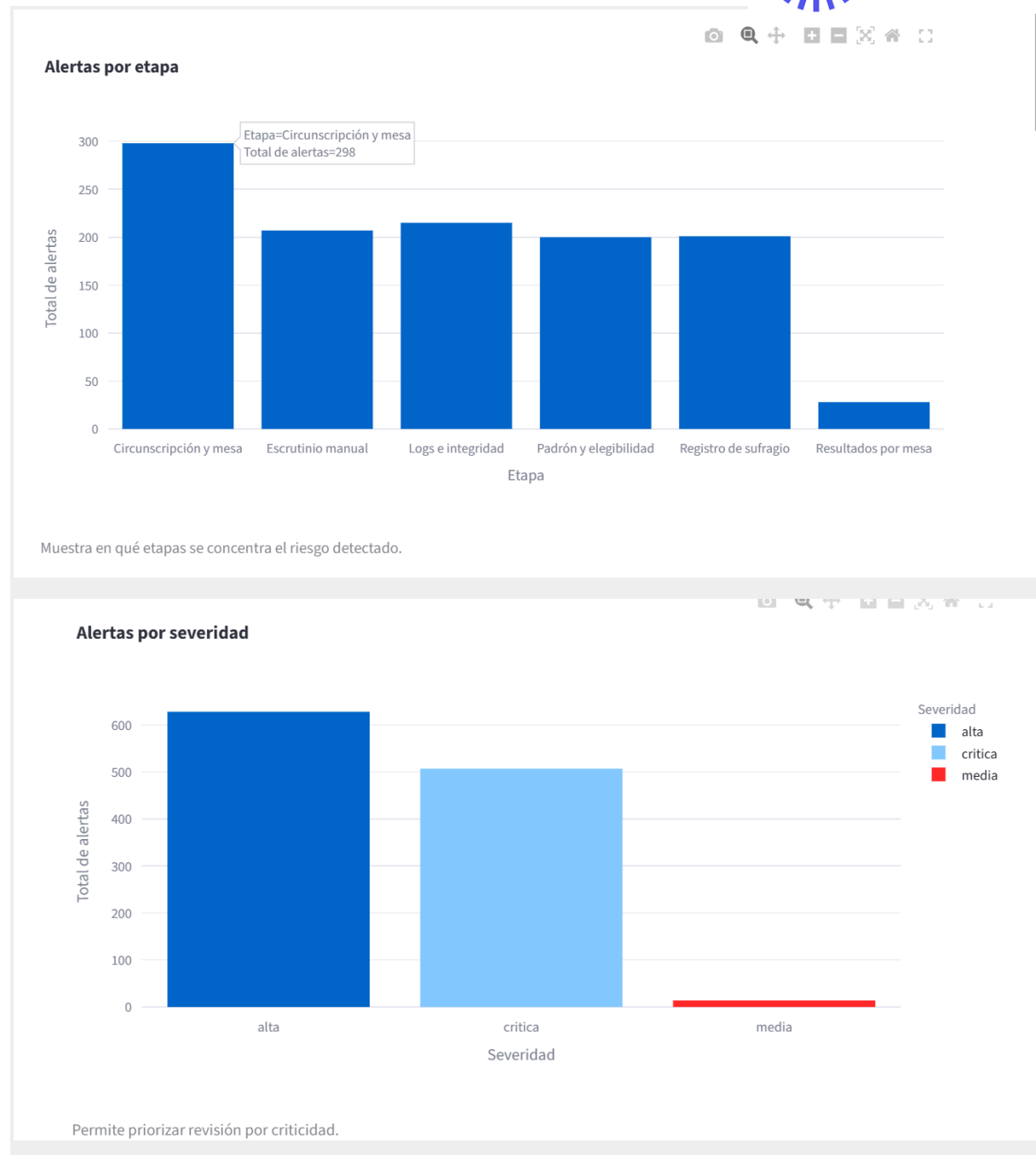
Datasets cargados **9** Registros analizados **18960** Alertas detectadas **1149** Alertas críticas **507** Score general **2243** Validación **Exitosa**

	metrica	valor
0	alertas_totales	1149
1	alertas_criticas	507
2	entidades_en_ranking	576
3	alertas_esperadas	1169
4	alertas_detectadas	1141
5	coincidencias	767
6	precision_aproximada	0.6722
7	cobertura_aproximada	0.6561

4.2 Distribución de alertas por etapa

Etapa electoral	Alertas	% del total
Circunscripción y mesa	298	25.7%
Logs e integridad	224	19.3%
Escrutinio manual	207	17.9%
Registro de sufragio	201	17.4%
Padrón y elegibilidad	200	17.3%
Resultados por mesa	28	2.4%
Total	1,158	100%

La etapa de Circunscripción y mesa concentra el mayor volumen de alertas (25.7%), principalmente por la condición CIR-03 (tipo de boleta incorrecto, 103 alertas) y CIR-01 (voto fuera de mesa asignada, 102 alertas). La etapa de Resultados por mesa genera el menor número de alertas absolutas (28), lo que refleja que sus reglas operan sobre 80 mesas en lugar de sobre el conjunto de votantes individuales o eventos de log.

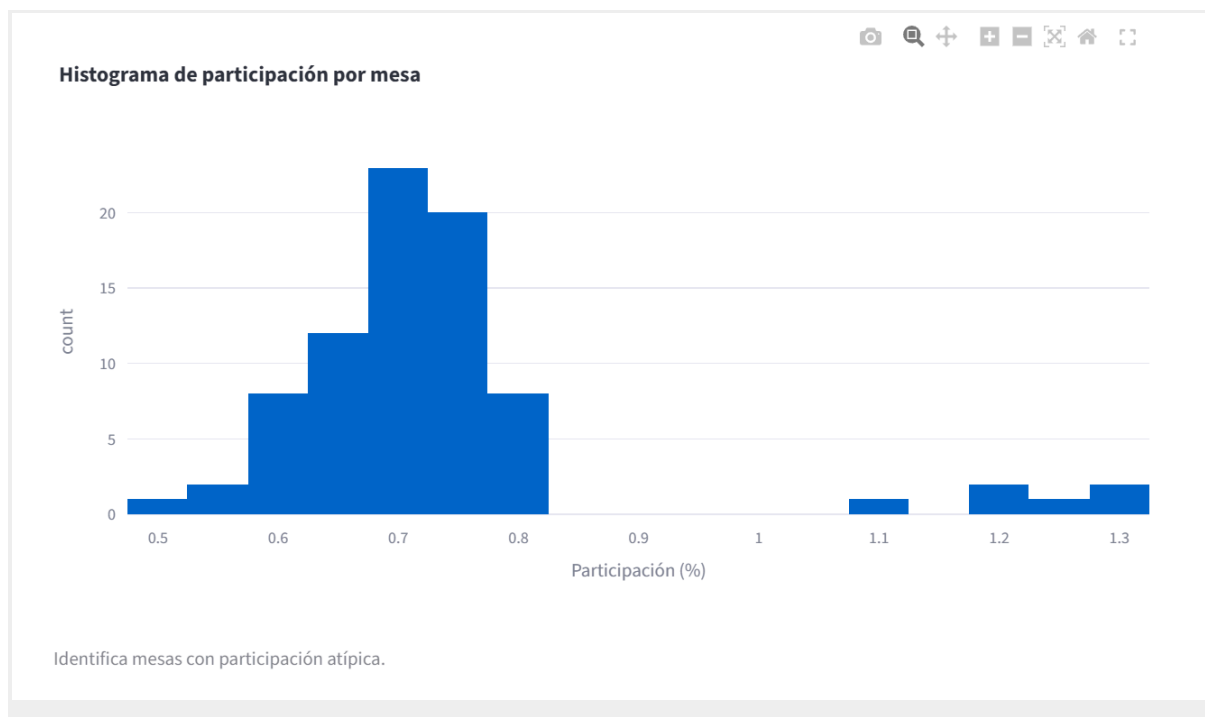


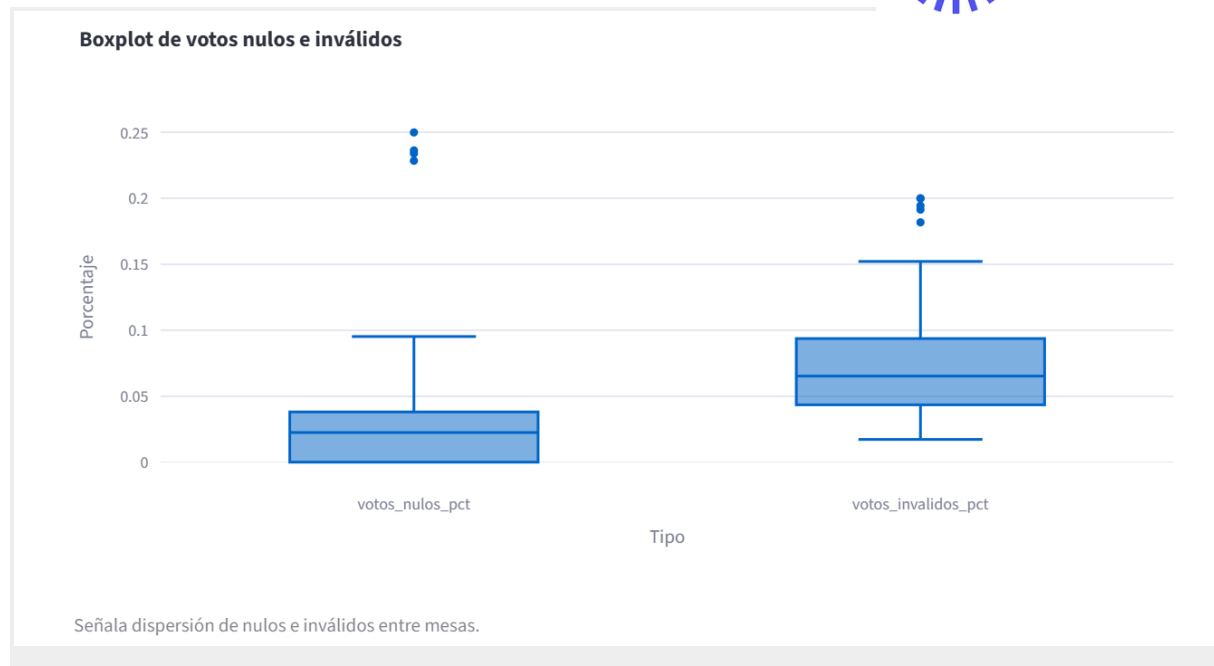
4.3 Alertas por código — Top 10

Código	Descripción	Cantidad
MAN-01	Voto válido declarado inválido por jurado	173
CIR-03	Tipo de boleta incorrecto para circunscripción	103
CIR-01	Voto registrado en mesa diferente a la asignada	102
SUF-02	Check-in de persona no habilitada en padrón	100

Código	Descripción	Cantidad
CIR-02	Voto registrado en circunscripción no autorizada	93
LOG-02	Acción ejecutada fuera de las permitidas por el rol (violación RBAC)	86
SUF-03	Check-in fuera del horario electoral	68
PAD-03	Edad imposible o superior a 115 años	50
PAD-06	Documento de identidad duplicado en el padrón	50
PAD-01	Votante fallecido con sufragio registrado	35

El código MAN-01 (voto válido declarado inválido) es el más frecuente con 173 instancias, lo que refleja la densidad de anomalías inyectadas en el proceso de escrutinio manual. Los 86 casos de LOG-02 (violaciones de RBAC) muestran que el control de acceso basado en roles es el vector de ataque más explotado en la capa de operaciones del sistema, consistente con los patrones documentados en Kent y Souppaya (2006) sobre el monitoreo de eventos de acceso no autorizado.



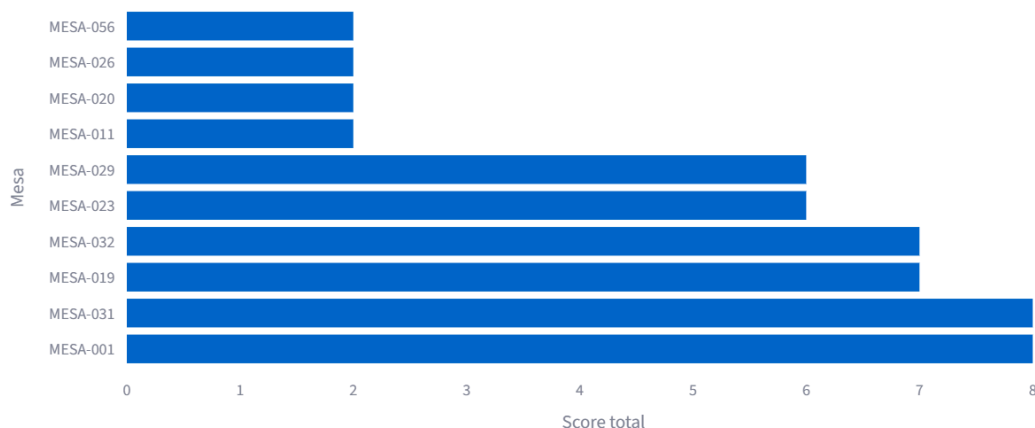


4.4 Ranking de entidades de riesgo — Top 10

Entidad	ID	Score total	Alertas	Clasificación
usuario_id	USR-0041	19	10	Revisión prioritaria
voter_id	V001825	11	4	Riesgo alto
voter_id	V003871	11	4	Riesgo alto
mesa_id	MESA-001	8	4	Riesgo alto
mesa_id	MESA-031	8	4	Riesgo alto
archivo_id	ARC-000068	8	3	Riesgo alto
voter_id	V000876	8	3	Riesgo alto
voter_id	V001865	8	3	Riesgo alto
voter_id	V001965	8	3	Riesgo alto
voter_id	V002133	8	3	Riesgo alto

El usuario USR-0041 es la única entidad clasificada como "Revisión prioritaria" con un score acumulado de 19 puntos proveniente de 10 alertas. Este perfil — múltiples alertas de alta severidad concentradas en un solo usuario — es consistente con el patrón de una cuenta con permisos excesivos o potencialmente vulnerada. Las mesas MESA-001 y MESA-031 presentan 4 alertas combinadas de severidades crítica y alta, posicionándolas como candidatas primarias para auditoría presencial (revisión física de actas y boletas de la mesa).

Top 10 mesas por score de riesgo



Prioriza las 10 mesas con mayor score de riesgo.

Ranking de riesgo

	entidad_tipo	entidad_id	score_total	total_alertas	max_severidad	clasificacion_riesgo
0	usuario_id	USR-0041	19	10	alta	Revisión prioritaria
1	voter_id	V001825	11	4	critica	Riesgo alto
2	voter_id	V003871	11	4	critica	Riesgo alto
3	mesa_id	MESA-001	8	4	critica	Riesgo alto
4	mesa_id	MESA-031	8	4	critica	Riesgo alto
5	voter_id	V000876	8	3	critica	Riesgo alto
6	voter_id	V001865	8	3	critica	Riesgo alto
7	voter_id	V001965	8	3	critica	Riesgo alto
8	voter_id	V002133	8	3	critica	Riesgo alto
9	mesa_id	MESA-019	7	3	critica	Riesgo medio

4.5 Análisis de las métricas de evaluación

La precisión de 67.0% y la cobertura de 65.8% son coherentes con el diseño del sistema y no representan un resultado inesperado. Hay dos factores que explican los valores obtenidos:

Factor 1 — El Z-score genera señales no etiquetadas en el ground truth. El ground truth sintético etiqueta únicamente las anomalías inyectadas de forma determinística durante la generación de datos. El Z-score puede marcar mesas que son estadísticamente atípicas en el conjunto pero que no fueron inyectadas como fraudulentas. Estas señales se cuentan como falsos positivos en el cálculo de precisión, aunque en un sistema de auditoría representan candidatos legítimos para revisión.

Factor 2 — No toda anomalía inyectada es detectada. La cobertura del 65.8% indica que aproximadamente una de cada tres anomalías del ground truth no es capturada. Esto es esperado

porque algunas anomalías inyectadas podrían tener Z-scores por debajo del umbral de 3 (no son suficientemente atípicas en el conjunto) o porque la regla determinística correspondiente no se activa en todos los casos dependiendo de la combinación de campos.

En el contexto de auditoría electoral, donde el objetivo es reducir el universo de revisión manual y priorizar entidades de riesgo — no reemplazar la auditoría humana — una cobertura de 65.8% significa que el sistema captura aproximadamente dos tercios de las anomalías conocidas, lo que es una reducción significativa del esfuerzo manual requerido. El U.S. Election Assistance Commission (2024) reconoce que ningún sistema automatizado de auditoría puede sustituir la verificación documental, técnica y contextual por auditores calificados.

5. Conclusiones

Deploy :

Resumen **Hallazgos** Tabla de alertas Evidencia y acciones

Anomalías Identificadas

A continuación se describen los hallazgos principales en lenguaje natural:

INT-01: Se detectaron 20 casos críticos donde hash original diferente del hash actual. Estas anomalías afectan a 20 entidades únicas y requieren atención inmediata.

INT-02: Se detectaron 9 casos críticos donde reporte publicado diferente del reporte firmado. Estas anomalías afectan a 9 entidades únicas y requieren atención inmediata.

LOG-01: Se detectaron 19 casos críticos donde modificación posterior al cierre sin aprobación. Estas anomalías afectan a 19 entidades únicas y requieren atención inmediata.

LOG-02: Se detectaron 80 casos críticos donde acción no permitida por el rol del usuario. Estas anomalías afectan a 80 entidades únicas y requieren atención inmediata.

LOG-05: Se detectaron 27 casos críticos donde hash modificado en evento no autorizado. Estas anomalías afectan a 27 entidades únicas y requieren atención inmediata.

5.1 Conclusiones técnicas

El proyecto demuestra que es posible construir un sistema de detección de anomalías electorales funcional, extensible y evaluable con herramientas de análisis de datos estándar (Python, pandas, SciPy) sin recurrir a técnicas de aprendizaje automático.

- **Las reglas determinísticas detectan con precisión las violaciones lógicas absolutas:** votantes fallecidos con sufragio registrado, desbordamiento de conteos de mesa y violaciones de RBAC son inequívocos y no requieren contexto estadístico para su identificación.

- **El Z-score detecta anomalías que las reglas no pueden capturar:** la supresión estadística de votos o la carga anómala de un operador son invisibles para reglas fijas pero emergen como outliers cuando se analizan en el contexto del conjunto.
- **La arquitectura modular por capas con el patrón Strategy** permite agregar nuevos detectores (para nuevas etapas o nuevas regulaciones) sin modificar el pipeline ni los componentes existentes, cumpliendo el principio abierto/cerrado.
- **La función de puntuación ponderada por severidad** transforma 1,158 alertas individuales en un ranking de 582 entidades ordenadas por riesgo acumulado, permitiendo al auditor identificar las 10 entidades prioritarias con un costo de revisión acotado.

5.2 Conclusiones de ciberseguridad

- **RBAC sin monitoreo es insuficiente:** el sistema detecta no solo la existencia de roles incorrectos sino las acciones concretas ejecutadas fuera del rol (86 instancias de LOG-02), lo que permite una respuesta forense precisa.
- **La separación de funciones (SoD) es crítica en sistemas electorales:** un operador que modifica y aprueba sus propias acciones representa un riesgo sistémico que el control LOG-06 detecta de forma explícita, sin depender de que el resto del sistema haya fallado.
- **La integridad de archivos post-cierre es el punto de mayor exposición en la capa de evidencia digital:** INT-01 detecta modificaciones de hashes que indican alteración de actas, el vector más directo de manipulación de resultados.
- **El comportamiento de operadores como señal de compromiso:** USR-0041 (score=19, 10 alertas) es detectable incluso si cada acción individual parece aisladamente justificable; es el patrón agregado el que revela el riesgo de una cuenta potencialmente vulnerada.

5.3 Limitaciones y trabajo futuro

- **Datos sintéticos exclusivamente:** los patrones de anomalías inyectadas no representan la distribución de irregularidades en procesos electorales reales. Un paso siguiente sería calibrar los umbrales del sistema con datos históricos anonimizados de auditorías certificadas.
- **Sin PKI real:** implementar firma digital criptográfica con certificados X.509 y HSMs elevaría el modelo de integridad de conceptual a criptográficamente verificable, cumpliendo estándares de evidencia digital forense.
- **Sin cifrado en reposo ni en tránsito:** un entorno productivo requeriría cifrado TLS para la interfaz web y cifrado en disco para los datasets, conforme a las prácticas de desarrollo seguro del NIST SSDF (Scarfone et al., 2022).
- **Sin integración con SIEM:** el sistema detecta y prioriza, pero no dispara flujos automáticos de respuesta ni correlaciona eventos en tiempo real. La integración con plataformas de gestión de eventos de seguridad permitiría detección incremental durante la jornada electoral, no solo post-proceso.

5.4 Reflexión final

El Electoral Integrity Analyzer ilustra cómo los principios de ciberseguridad operacional (RBAC, SoD, integridad por hash, trazabilidad de logs) pueden implementarse como controles verificables y auditables dentro de un sistema de análisis de datos. La combinación de reglas determinísticas y

Z-score cubre dos vectores complementarios de irregularidad: el evidente (violación lógica) y el sutil (desviación estadística significativa). La arquitectura modular garantiza que el sistema puede evolucionar — nuevos detectores, nuevas etapas, nuevas regulaciones — sin comprometer los componentes existentes.

El patrón arquitectónico y los controles implementados son transferibles a dominios donde la integridad de datos críticos deba verificarse sistemáticamente: auditoría financiera, registros médicos, cadenas de suministro o sistemas de control industrial, siempre que los activos, los roles autorizados y las transiciones válidas sean definibles con precisión.

6. Referencias

- Cavoukian, A. (2009). Privacy by design: The 7 foundational principles. Information and Privacy Commissioner of Ontario.
- Clark, D. D., & Wilson, D. R. (1987). A comparison of commercial and military computer security policies. En Proceedings of the 1987 IEEE Symposium on Research in Security and Privacy (pp. 184–193). IEEE. <https://doi.org/10.1109/SP.1987.10001>
- Hu, V., Ferraiolo, D., & Kuhn, R. (2014). Guide to attribute based access control (ABAC) definition and considerations (NIST SP 800-162). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-162>
- Kent, K., & Souppaya, M. (2006). Guide to computer security log management (NIST SP 800-92). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-92>
- McKinney, W. (2010). Data structures for statistical computing in Python. En Proceedings of the 9th Python in Science Conference, 445, 51–56. <https://doi.org/10.25080/Majora-92bf1922-00a>
- National Institute of Standards and Technology. (2015). Secure hash standard (NIST FIPS 180-4). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.FIPS.180-4>
- National Institute of Standards and Technology. (2024a). NIST common data formats for election systems (NIST GCR 24-058). U.S. Department of Commerce. <https://nvlpubs.nist.gov/nistpubs/gcr/2024/24-058/NIST.GCR.24-058.html>
- Plotly Technologies Inc. (2015). Collaborative data science [Software]. <https://plot.ly>
- Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. Proceedings of the IEEE, 63(9), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>
- Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. IEEE Computer, 29(2), 38–47. <https://doi.org/10.1109/2.485845>
- Scarfone, K., Souppaya, M., & Dodson, D. (2022). Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities (NIST SP 800-218). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-218>
- Shewhart, W. A. (1931). Economic control of quality of manufactured product. Van Nostrand.
- Streamlit Inc. (2019). Streamlit: The fastest way to build data apps [Software]. <https://streamlit.io>
- U.S. Election Assistance Commission. (2024). Post-election tabulation audits: A practical guide for beginners and beyond. https://www.eac.gov/sites/default/files/2024-11/Post_Election_Tabulation_Audit_Guide_508.pdf
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... van Mulbregt, P. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17, 261–272. <https://doi.org/10.1038/s41592-019-0686-2>